

Rappresentazioni lineari, superposizione e cybersecurity

Materiale di orientamento per la scelta di un argomento di tesi triennale

Jesús F. Cevallos-Moreno | Autunno 2026

jf.cevallosmoreno@uninsubria.it

<http://www.dista.uninsubria.it/~jesus.cevallos/>

Come leggere questo documento

Questo documento non contiene una proposta unica: è una mappa. Descrive un filone di ricerca — come le reti neurali rappresentano i concetti, e come si potrebbe portare questa domanda nel dominio della sicurezza informatica — indica dove quel filone si rompe se lo si affronta ingenuamente, e da quelle rotture estrae cinque domande di ricerca separate, ciascuna delle quali può essere una tesi triennale a sé stante.

Le sezioni 1 e 2 introducono il quadro concettuale partendo da zero: non serve alcun background di machine learning, e i termini tecnici sono raccolti nel glossario finale, che abbiamo cercato di rendere leggibile anche a chi incontra questi concetti per la prima volta. La sezione 3 elenca in forma compatta gli ostacoli alla formulazione ingenua: serve solo perché ciascun ostacolo genera una delle domande della sezione 4. La sezione 4 è il cuore del documento e contiene le cinque proposte. La sezione 5 è il glossario, e la sezione 6 raccoglie i riferimenti bibliografici citati nel testo, con il titolo completo di ciascun lavoro.

Se volete andare al sodo: leggete la sezione 2, poi la sezione 4, poi tornate indietro su ciò che non è chiaro.

Indice

1	Il punto di partenza: come una rete neurale rappresenta l'informazione	3
1.1	Rappresentazioni distribuite	3
1.2	L'ipotesi di rappresentazione lineare e il principio di superposizione	3
1.3	Il modello matematico	4
1.4	Perché tutto questo interessa	6

2	Trasportare l'idea nella cybersecurity	6
2.1	Il dizionario di traduzione	6
2.2	Che cos'è MITRE ATT&CK	6
2.3	Che cos'è Caldera	7
2.4	L'ipotesi da testare	7
3	Sei ostacoli, in breve	7
3.1	L'aggregazione rende il risultato vero per costruzione	8
3.2	ATT&CK non è una base	8
3.3	Gli attacchi sono programmi, non scene	8
3.4	Il teorema di identificabilità non si applica direttamente a un IDS	9
3.5	g non è iniettiva	9
3.6	Igiene sperimentale	9
4	Le cinque proposte	10
4.1	Tesi A — L'additività è una proprietà del modello o dell'aggregazione?	10
4.2	Tesi B — Quali tecniche ATT&CK sono linearmente decodificabili dal traffico? . . .	11
4.3	Tesi C — Invarianza all'ordine e sue conseguenze di sicurezza	12
4.4	Tesi D — Il fine-tuning supervisionato distrugge la geometria compositiva?	12
4.5	Tesi E — Un banco di prova per la novità compositiva	13
4.6	Come orientarsi fra le cinque	14
5	Glossario	15
6	Riferimenti bibliografici	20

1 Il punto di partenza: come una rete neurale rappresenta l'informazione

Prima di tutto, cos'è una rete neurale? È un programma che impara da esempi, invece di essere scritto a mano istruzione per istruzione. Le si mostrano molti esempi (migliaia di immagini di elefanti, milioni di frasi, milioni di pacchetti di rete etichettati come "normali" o "sospetti") e, aggiustando progressivamente un gran numero di parametri interni, il programma impara da solo a riconoscere pattern. Non è necessario capire i dettagli di come avvenga questo apprendimento (la tecnica si chiama *gradient descent*, "discesa del gradiente": in breve, il programma corregge i propri parametri un po' alla volta, nella direzione che riduce l'errore commesso sugli esempi) per seguire questo documento. Ciò che ci interessa qui è cosa succede *dentro* la rete mentre elabora un input, dopo che l'apprendimento è terminato.

1.1 Rappresentazioni distribuite

Quando una rete neurale già addestrata elabora un'immagine, un testo o un pacchetto di rete, ciò che accade internamente è che l'input viene trasformato in un vettore di numeri reali — cioè, semplicemente, una lista ordinata di qualche centinaio o qualche migliaio di numeri. Questo vettore si chiama *rappresentazione* (o *embedding*, o *stato nascosto*) ed è tutto ciò che la rete "sa" dell'input in quel punto della computazione: non conserva l'immagine originale, conserva solo questi numeri.

La domanda naturale è: cosa significano quei numeri? Storicamente ci sono state due risposte opposte. Nella codifica locale ogni singola unità di calcolo della rete (un "neurone", cioè uno dei tanti piccoli componenti che insieme formano la rete) corrisponde a un concetto: il neurone numero 42 si accende se e solo se nell'immagine c'è un elefante, un po' come un interruttore dedicato. Nella codifica distribuita, invece, un concetto non vive in un singolo neurone ma in un pattern di attivazione su molti neuroni contemporaneamente, e ciascun neurone partecipa a molti concetti diversi — un po' come in un'orchestra, dove nessuno strumento da solo "è" la melodia, ma la melodia emerge dalla combinazione di tutti. L'evidenza empirica, sia nelle reti artificiali sia nel cervello biologico, punta decisamente verso la seconda ipotesi.

Questo crea un problema pratico immediato: se il concetto "elefante" non sta in un neurone ma è spalmato su cinquecento, come facciamo a leggerlo? È la domanda centrale del campo che si chiama *interpretabilità*: capire cosa "pensa" davvero una rete neurale guardando dentro i suoi numeri, invece di limitarsi a osservarne l'output finale.

1.2 L'ipotesi di rappresentazione lineare e il principio di sovrapposizione

C'è un fatto empirico sorprendente che rende il problema trattabile. Chiamiamo g la funzione che genera i dati a partire dai concetti (per esempio: un modello generativo che, a partire dalla descrizione testuale "un elefante", produce l'immagine di un elefante) e f la funzione calcolata dalla rete che analizza i dati (per esempio: un modello di visione che, data l'immagine, produce il vettore di rappresentazione descritto sopra). Entrambe queste funzioni sono, prese singolarmente, fortemente non lineari — cioè il loro comportamento non è affatto "proporzionale" o prevedibile per semplice somma: piccoli cambiamenti nell'input possono produrre effetti sproporzionati o inattesi nell'output, e non esiste una formula semplice che leghi input e output.

Eppure, se si prende la rappresentazione di un elefante e la si somma a quella di una palla rosa, si ottiene qualcosa di molto vicino alla rappresentazione dell'immagine che contiene entrambi gli oggetti:

$$f(g(\text{"elefante"})) + f(g(\text{"palla rosa"})) \approx f(g(\text{"un elefante e una palla rosa"}))$$

Questa proprietà si chiama additività: sommare le rappresentazioni equivale (circa) a rappresentare la somma dei concetti. Insieme all'omogeneità (moltiplicare il concetto per un numero moltiplica la rappresentazione per lo stesso numero) essa è la definizione stessa di funzione lineare, cioè di una funzione che si comporta "bene" rispetto a somme e moltiplicazioni, nel modo più semplice possibile. Quindi la composizione $f \circ g$ — il percorso completo dal concetto astratto (l'idea di "elefante") alla rappresentazione interna (il vettore di numeri) — si comporta come una funzione lineare, pur essendo ottenuta componendo due funzioni che, singolarmente, sono entrambe non lineari e complicate. Questo fatto sorprendente si chiama ipotesi di rappresentazione lineare.

La stessa proprietà, vista da un'altra angolazione, è ciò che rende possibili le note "analogie vettoriali" negli embedding di parole (l'esempio classico, reso famoso da alcuni dei primi modelli di linguaggio, è "re" - "uomo" + "donna" $\approx$ "regina"):

$$f(g(\text{"palla rosa"})) - f(g(\text{"palla"})) \approx f(g(\text{"elefante rosa"})) - f(g(\text{"elefante"}))$$

Entrambi i membri di questa equazione approssimano una direzione nello spazio delle rappresentazioni che corrisponde al concetto "rosa", indipendentemente dall'oggetto a cui è attaccato: è come se "rosa" fosse una freccia fissa che si può sommare a qualunque oggetto per colorarlo. Il termine sovrapposizione indica esattamente questo fenomeno: molti concetti sono sovrapposti linearmente nello stesso spazio di rappresentazione, usando ciascuno una propria "direzione", e — punto cruciale — queste direzioni non sono perpendicolari fra loro (in gergo: non sono ortogonali), perché i concetti da rappresentare sono molti di più delle dimensioni disponibili nel vettore. È un po' come cercare di sistemare mille etichette diverse su una scatola che ha spazio per scriverne solo cento: la rete ci riesce comunque, ma deve farle sovrapporre parzialmente.

Tutto il quadro concettuale di questa sezione e della prossima (ipotesi di rappresentazione lineare, sovrapposizione, e il modello a tre passi che segue) è esposto in modo sistematico e accessibile in Klindt, O'Neill, Reizinger, Maurer e Miolane (2025), *"From Superposition to Sparse Codes: Interpretable Representations in Neural Networks"* — riferimento [1] in bibliografia, § 6. È la lettura di riferimento più utile per chi voglia approfondire prima di scegliere una tesi.

1.3 Il modello matematico

Formalizziamo, con l'obiettivo di essere precisi ma senza dare per scontato nulla. Chiamiamo z il vettore delle variabili latenti, cioè delle quantità che non osserviamo direttamente ma che immaginiamo abbiano generato il dato: le sue componenti sono i concetti presenti nel mondo (per esempio, la componente numero 42 di z vale 1 se nella scena c'è un elefante, 0 altrimenti). Il numero totale di concetti possibili, che chiamiamo N , è enorme, ma in una scena concreta solo pochissime componenti di z sono diverse da zero: diciamo che z è sparso (un vettore sparso è, appunto, un vettore fatto quasi tutto di zeri, con solo poche eccezioni "accese"). Chiamiamo g la funzione generativa che produce il dato osservato x a partire da z , e f la rete neurale che, a partire da x , produce la rappresentazione y . Componendo le due, otteniamo $h = f \circ g$. Il punto chiave è che la rappresentazione y ha molte meno dimensioni (chiamiamo questo numero M) dei concetti N che

deve in qualche modo contenere: M è molto più piccolo di N . Il quadro teorico si articola allora in tre passi.

1. Identificabilità. "Identificabilità" è la proprietà per cui, guardando solo il risultato finale, si può risalire con certezza (o quasi) a quali fossero i concetti di partenza. Un risultato teorico recente (Klindt, O'Neill, Reizinger, Maurer e Miolane, 2025 — *"From Superposition to Sparse Codes: Interpretable Representations in Neural Networks"*, riferimento [1] in bibliografia, § 6) dimostra che, sotto certe ipotesi, una rete addestrata a classificare oggetti — cioè a dire "questa immagine contiene un elefante", "questa no" — con un particolare criterio di addestramento (la cross-entropy, una misura di quanto le previsioni della rete si discostano dalla risposta corretta) e portata al punto in cui l'errore commesso è il più basso possibile, finisce per calcolare una funzione $h = \text{fog}$ che è lineare. In parole semplici: la rete impara, come "sottoprodotto" dell'addestramento, a invertire il processo che genera il mondo a partire dai concetti, a meno di una semplice trasformazione lineare (una rotazione o un riscaldamento dello spazio, per intenderci). È il motivo teorico per cui ci si aspetta l'additività osservata al § 1.2 — cioè una spiegazione matematica del perché la somma di due rappresentazioni "funziona".

Le ipotesi di questo teorema sono piuttosto forti (richiedono condizioni ideali che raramente si verificano del tutto nella pratica), ed è proprio lì che l'edificio scricchiola: ne parliamo al § 3.4.

2. Compressed sensing e sparse coding. Se h è lineare, esiste una matrice (una tabella di numeri che rappresenta una trasformazione lineare) Φ , di dimensioni $M \times N$, tale che $y = \Phi z$. Abbiamo M equazioni ma N incognite, con N maggiore di M : il sistema è sottodeterminato, cioè in linea di principio non ha una soluzione unica — troppe incognite per il numero di equazioni disponibili. Ma se z è sparso — al più K componenti diverse da zero, con K molto più piccolo di N — una teoria matematica chiamata compressed sensing ("campionamento compresso") garantisce che z si possa comunque recuperare con alta probabilità, purché il numero di dimensioni disponibili M sia sufficientemente grande rispetto a K e a N : precisamente, deve valere all'incirca $M > K \cdot \log(N / K)$ (dove $\log$ indica il logaritmo, una funzione che cresce molto più lentamente del suo argomento: raddoppiare N non raddoppia il numero di misure necessarie, ne aggiunge solo una piccola quantità).

Il recupero pratico si fa con lo sparse coding ("codifica sparsa"): si cerca un "dizionario" di riferimento Θ (un'altra matrice) e dei "codici" $\hat{z}$ il più possibile sparsi (con pochi elementi diversi da zero) che ricostruiscano y usando solo poche voci del dizionario. Si fa minimizzando una quantità che bilancia due obiettivi contrastanti: l'errore di ricostruzione $\|y - \Theta \hat{z}\|^2$ (quanto la ricostruzione somiglia all'originale) e un termine $\|\hat{z}\|_1$ che penalizza le soluzioni poco sparse, cioè spinge il risultato ad avere il minor numero possibile di componenti diverse da zero. La versione moderna ed efficiente di questa procedura è lo sparse autoencoder (SAE): invece di risolvere questa ottimizzazione da capo per ogni singolo dato, si addestra una volta per tutte una funzione che mappa direttamente y in $\hat{z}$.

3. Interpretabilità quantitativa. Le vere variabili latenti non le conosciamo mai con certezza assoluta (nessuno ci consegna un elenco ufficiale di "tutti i concetti presenti in questa immagine"), quindi non possiamo verificare in modo diretto di averle recuperate esattamente. Si usano allora metriche surrogate, cioè misure indirette che fungono da approssimazione ragionevole: tipicamente compiti psicofisici, come chiedere a una persona in carne e ossa di individuare l'intruso in un insieme di parole associate a uno stesso concetto estratto dalla rete. Queste metriche sono scelte in modo

da essere invarianti a permutazioni (cioè non cambiano se riordiniamo gli elementi), perché quella è esattamente l'ambiguità residua che la teoria stessa ammette come inevitabile.

1.4 Perché tutto questo interessa

Se questo programma di ricerca funziona, si ottiene una cosa notevole: dato lo stato interno di una rete neurale, lo si può scomporre in un elenco leggibile di concetti attivi ("c'è un elefante", "c'è del rosa"), invece di dover guardare un vettore opaco di 4096 numeri senza alcun significato apparente. È il progetto che sta dietro alla ricerca sull'interpretabilità dei moderni modelli linguistici (i sistemi come quelli usati dai chatbot). La domanda che ci interessa qui è: funziona anche altrove? In particolare, funziona in un dominio dove i concetti non sono elefanti e palle rosa, ma azioni ostili su una rete di calcolatori?

2 Trasportare l'idea nella cybersecurity

2.1 Il dizionario di traduzione

L'idea è di istanziare (cioè riempire con oggetti concreti) lo schema $z \rightarrow x \rightarrow y$ visto sopra, usando oggetti del dominio della sicurezza informatica al posto di elefanti e palle rosa:

Simbolo	Nel mondo delle immagini	Nel mondo della cybersecurity
z (i concetti nascosti)	oggetti nella scena: elefante, palla	tecniche di attacco secondo la tassonomia MITRE ATT&CK: scansione di rete, movimento laterale, esfiltrazione
g (il generatore)	modello generativo testo $\rightarrow$ immagine	Caldera: motore di emulazione che esegue realmente le tecniche in un laboratorio
x (il dato osservato)	l'immagine	il traffico di rete catturato (file pcap o flussi)
f (la rete neurale)	modello di visione (ViT)	un sistema di rilevamento delle intrusioni (IDS) basato su rete neurale
y (la rappresentazione)	embedding dell'immagine	stato nascosto dell'IDS mentre osserva quel traffico

2.2 Che cos'è MITRE ATT&CK

ATT&CK è una tassonomia pubblica (cioè un catalogo organizzato e condiviso) dei comportamenti degli attaccanti informatici, curata dalla MITRE Corporation, un'organizzazione statunitense senza scopo di lucro attiva nella ricerca sulla sicurezza. Non descrive vulnerabilità tecniche specifiche di questo o quel software, ma cosa fa concretamente un attaccante una volta entrato in un sistema. È organizzato in tattiche (l'obiettivo che l'attaccante persegue in quel momento: "Discovery" cioè

esplorare la rete, "Lateral Movement" cioè spostarsi verso altre macchine, "Exfiltration" cioè portare fuori i dati rubati) e tecniche (il come concreto: per esempio "T1046 – Network Service Discovery" o "T1021.001 – Remote Desktop Protocol"). È il vocabolario standard con cui l'industria della sicurezza descrive gli attacchi, un po' come un manuale condiviso di "mosse" possibili. Nel nostro schema è il candidato naturale per il ruolo di z , cioè dei concetti nascosti da recuperare.

2.3 Che cos'è Caldera

Caldera è una piattaforma open source (a codice sorgente pubblico e liberamente modificabile) di emulazione dell'avversario: si allestisce un piccolo laboratorio di macchine virtuali (computer simulati via software, isolati e sicuri da usare per questi esperimenti), si installa un agente software su di esse, e si dice al sistema "esegui la tecnica T1046 su questa macchina". Caldera la esegue davvero, generando traffico di rete reale e osservabile, non simulato a parole. Ogni singola azione eseguibile si chiama ability (letteralmente "abilità") ed è etichettata con l'identificativo ATT&CK corrispondente. Nel nostro schema, Caldera gioca il ruolo di g : è la funzione che trasforma concetti astratti (le tecniche) in dati concreti (il traffico di rete). Nota di attualità: nel maggio 2026 MITRE ha donato Caldera alla Apache Software Foundation, dove il progetto prosegue come Apache Caldera (Incubating, cioè ancora in fase di incubazione all'interno della fondazione); la libreria pubblica di abilities disponibili è di conseguenza un po' più ristretta di quella storica.

2.4 L'ipotesi da testare

A questo punto l'ipotesi centrale si scrive quasi da sola. Prendiamo due tecniche z_1 e z_2 , le eseguiamo separatamente in laboratorio, catturiamo il traffico generato da ciascuna, lo diamo in pasto all'IDS (il sistema di rilevamento) e otteniamo le due rappresentazioni y_1 e y_2 . Poi eseguiamo un attacco z_3 che è la combinazione concettuale delle due tecniche precedenti (per esempio: prima scansione la rete, poi mi sposto lateralmente), e otteniamo la rappresentazione y_3 . La domanda è se vale, anche solo approssimativamente, la stessa proprietà di additività vista con elefanti e palle:

$$f(g(z_1)) + f(g(z_2)) \approx f(g(z_3)) ?$$

Se la risposta fosse sì, le conseguenze sarebbero interessanti: significherebbe che lo stato interno di un IDS ammette una scomposizione in direzioni corrispondenti a singole tecniche d'attacco, e quindi che si potrebbe passare da un allarme binario ("c'è un attacco", sì o no) a una spiegazione strutturata e leggibile da un essere umano ("sono attive le tecniche T1046 e T1021"). Questo colmerebbe quello che in letteratura si chiama semantic gap, cioè il divario fra ciò che un rilevatore automatico segnala in modo criptico e ciò di cui una persona ha realmente bisogno per decidere come reagire.

3 Sei ostacoli, in breve

La formulazione appena data, presa alla lettera, non produrrebbe un risultato affidabile: ci sono almeno sei motivi per diffidarne. Li elenchiamo qui in forma compatta, senza approfondirli troppo: servono solo perché ciascuno di essi genera una delle domande di ricerca della sezione 4, e leggendo quelle proposte è utile sapere da dove nascono.

3.1 L'aggregazione rende il risultato vero per costruzione

Il traffico di rete non è un oggetto singolo come un'immagine: è una sequenza nel tempo (pacchetti, flussi, finestre temporali). Per ottenere un unico vettore y su cui ragionare bisogna aggregare tutta questa sequenza in qualche modo, e in pratica ciò significa quasi sempre farne la media aritmetica. Ma se la finestra temporale di z_1 contiene n_1 flussi con media μ_1 e quella di z_2 ne contiene n_2 con media μ_2 , la finestra combinata contiene semplicemente i flussi di entrambe, e quindi la sua media è automaticamente

$$f(g(z_3)) \approx (n_1\mu_1 + n_2\mu_2) / (n_1 + n_2)$$

cioè una combinazione convessa (una media pesata) di μ_1 e μ_2 , che si colloca "fra" i due vettori originali e ha quindi una cosine similarity alta con la loro somma — e questo è vero indipendentemente da f , persino con una rete i cui pesi siano scelti a caso e mai addestrati. In altre parole: misureremmo semplicemente una proprietà aritmetica della media, non una proprietà intelligente della rete. A complicare le cose, gli spazi di embedding del traffico di rete sono in pratica fortemente anisotropi (i vettori tendono a concentrarsi tutti in una zona ristretta dello spazio invece di distribuirsi uniformemente: analisi recenti riportano una cosine similarity media intorno a 0,64 anche fra flussi scelti a caso e senza relazione fra loro, nel migliore dei modelli testati). Si capisce quindi che, senza un termine di paragone calibrato con cura, un valore osservato di 0,96 non ci dice nulla di per sé.
→ Tesi A.

3.2 ATT&CK non è una base

La teoria richiede che le variabili z siano sorgenti distinte e, idealmente, statisticamente indipendenti fra loro. ATT&CK, però, non ha questa struttura: è un grafo gerarchico (per esempio la tecnica T1059 è "genitore" della tecnica più specifica T1059.001), è una tassonomia negoziata da analisti umani nel tempo e non una scomposizione naturale e oggettiva del traffico di rete, e le sue tecniche co-occorrono (compaiono insieme) in modo fortemente strutturato, perché gli attacchi seguono sequenze logiche tipiche. In più, ciò che Caldera fa effettivamente variare non è la tecnica astratta ma la singola ability concreta: la stessa tecnica realizzata con lo strumento `nmap` oppure con uno script PowerShell produce traffico di rete completamente diverso, quindi ciò che in pratica si recupererebbe non sarebbe la tecnica in sé, ma lo strumento specifico usato per eseguirla.

3.3 Gli attacchi sono programmi, non scene

In Caldera l'esecuzione combinata di più tecniche passa attraverso un pianificatore automatico e un "magazzino di fatti": l'ability B parte solo quando l'ability A ha prodotto le informazioni che B richiede per funzionare. "Eseguiare z_1 e z_2 insieme" significa quindi eseguire un piccolo programma con dipendenze precise fra i suoi passi, in cui ogni parte condiziona bersagli e tempi dell'altra — non è come mettere semplicemente due oggetti fianco a fianco in una fotografia. La non additività, in questo caso, viene iniettata direttamente dal generatore g , non scoperta a posteriori nella rappresentazione. Da qui nasce il cosiddetto problema del binding (letteralmente "legame", "associazione"): una rappresentazione puramente additiva è per costruzione invariante all'ordine e a chi-ha-fatto-cosa — la somma di "blu", "triangolo", "rosso", "quadrato" è identica sia che il triangolo sia blu e il quadrato rosso, sia il contrario. Tradotto nel nostro dominio:

Se la rappresentazione interna di un IDS è additiva rispetto alle tecniche, allora è cieca all'ordine di

esecuzione e a quale macchina abbia fatto cosa: "prima ricognizione poi esfiltrazione" ed "prima esfiltrazione poi ricognizione" finirebbero nello stesso identico punto dello spazio delle rappresentazioni. → Tesi C.

3.4 Il teorema di identificabilità non si applica direttamente a un IDS

Il teorema di identificabilità citato al § 1.3 richiede tre condizioni precise: che l'addestramento abbia raggiunto il minimo globale della cross-entropy (il punto di errore più basso in assoluto, non solo un buon compromesso), che il classificatore finale sia esso stesso lineare, e che le dimensioni delle variabili latenti e della rappresentazione siano uguali. Nessuna delle tre condizioni vale davvero per un IDS reale allenato su dati reali. C'è però anche un problema più quantitativo e più sottile, chiamato neural collapse ("collasso neurale", descritto da Papayan, Han e Donoho nel 2020 in *"Prevalence of Neural Collapse during the Terminal Phase of Deep Learning Training"*, riferimento [2] in bibliografia, § 6): nella fase finale dell'addestramento di un classificatore, le rappresentazioni tendono a collassare verso una configurazione geometrica molto semplice, con tante "punte" quante sono le classi da distinguere (un simpleso con K vertici, dove K è il numero di classi), e la variabilità interna a ciascuna classe tende ad azzerarsi. In pratica, la dimensione effettivamente utile della rappresentazione si riduce a circa $K - 1$: con 5 classi da distinguere, restano solo quattro dimensioni realmente informative, anche se il vettore ne ha centinaia. Applicando il conto del § 1.3, con circa $N \approx 200$ tecniche possibili e $K = 3$ attive contemporaneamente, servirebbe $M \gtrsim 3 \cdot \log(67) \approx 12,6$ dimensioni utili — e questo prima ancora di considerare altre costanti che nella pratica peggiorano la stima. Chiedere a uno spazio di rango 4 di contenere 200 concetti sovrapposti è chiedere troppo. La correzione più promettente è usare come f un modello fondazionale per il traffico di rete, addestrato in modo auto-supervisionato (cioè senza bisogno di etichette scritte a mano, si veda il glossario), che tende a mantenere una geometria interna più ricca e meno "collassata". → Tesi D.

3.5 g non è iniettiva

Una funzione si dice iniettiva se a input diversi corrispondono sempre output diversi — qui, " g non è iniettiva" significa che tecniche di attacco diverse possono produrre lo stesso identico traffico di rete osservabile, rendendole indistinguibili da quel solo canale. E infatti una porzione consistente delle tecniche di ATT&CK non lascia alcuna firma visibile nel traffico di rete: l'iniezione di codice, l'offuscamento di file, gran parte delle tecniche di evasione sono semplicemente invisibili guardando un pcap. Se $g(z_i) = g(z_j)$ sul canale di rete per molte coppie di tecniche diverse, allora la funzione h non è invertibile e le garanzie teoriche della sezione 1 svaniscono per quelle tecniche. Serve quindi determinare empiricamente, con esperimenti mirati, l'insieme delle tecniche su cui questo intero discorso ha effettivamente senso. → Tesi B.

3.6 Igiene sperimentale

Un censimento molto citato nella comunità (Arp e colleghi, USENIX Security 2022, *"Dos and Don'ts of Machine Learning in Computer Security"*, riferimento [3] in bibliografia, § 6) analizza un gran numero di articoli su machine learning e sicurezza informatica e trova segnali di sampling bias (campioni non rappresentativi) e data snooping (informazione che "trapela" indebitamente dai dati di test in fase di addestramento) in oltre il 73% dei lavori esaminati, con almeno tre delle dieci trappole catalogate presenti in ciascuno di essi. L'esempio che gli autori usano per illustrare le correlazioni

spurie (cioè apparenti ma non reali) è esattamente il nostro caso: un IDS che impara a riconoscere l'intervallo di indirizzi IP usato dagli attaccanti nel laboratorio, invece di imparare a riconoscere gli attacchi in sé. In un laboratorio Caldera l'indirizzo IP dell'attaccante resta tipicamente costante per tutta la durata degli esperimenti: se non si randomizzano indirizzi, porte, tempi e traffico di sottofondo "innocente", la rete finisce per imparare le peculiarità del laboratorio invece del fenomeno che si voleva studiare.

Un avvertimento terminologico importante, da mettere per iscritto in qualunque tesi su questo tema: nella letteratura specialistica sugli IDS, da vent'anni, l'espressione "sparse autoencoder" indica un tipo di autoencoder sotto-completo (cioè con uno strato interno più piccolo dell'input) con una penalità di sparsità, usato per ridurre la dimensionalità dei dati. È l'esatto opposto dello sparse autoencoder sovra-completo dello sparse coding di cui parliamo in questo documento (§ 1.3), che ha invece uno strato interno più grande dell'input. È facile confondere i due se non si presta attenzione al contesto.

4 Le cinque proposte

Le critiche della sezione 3 non chiudono affatto il discorso: al contrario, lo frammentano in domande più piccole e meglio poste, ciascuna delle quali è affrontabile con gli strumenti e i tempi di una tesi triennale. Ognuna delle proposte che seguono ha la forma richiesta a una buona tesi sperimentale: un'ipotesi falsificabile (cioè che può essere messa alla prova e smentita, non solo confermata), un esperimento concreto che la mette alla prova, metriche quantitative per misurare il risultato, e un insieme dichiarato di assunzioni e limiti. Nessuna delle cinque richiede di produrre un'innovazione assoluta: richiede di essere affrontata con rigore.

Per ciascuna proposta indichiamo: la domanda di ricerca, perché è interessante, l'esperimento da condurre, le metriche da usare e una stima realistica della difficoltà.

4.1 Tesi A — L'additività è una proprietà del modello o dell'aggregazione?

Domanda. L'additività osservabile nelle rappresentazioni di traffico di rete è attribuibile davvero alla rete neurale, oppure è semplicemente un artefatto (un effetto collaterale non voluto) dell'operatore di aggregazione usato e della geometria anisotropa dello spazio, come discusso al § 3.1?

Perché è interessante. È una tesi di falsificazione, cioè mirata a mettere alla prova un'ipotesi per vedere se resiste, e le tesi di falsificazione sono spesso le più sottovalutate e in realtà le più solide dal punto di vista scientifico. Se la risposta è "è solo un artefatto", si è prodotto un risultato negativo pulito e utile, che vale da avvertimento per un intero filone di lavori futuri. Se la risposta è "no, l'effetto sopravvive anche ai controlli più severi", si è validato empiricamente il presupposto su cui poggia tutto il resto di questo programma di ricerca.

Esperimento. Si generano con Caldera esecuzioni di singole tecniche e di coppie di tecniche. Poi si misura la relazione additiva in quattro condizioni via via più severe, pensate come controlli progressivi:

1. con f = il modello IDS effettivamente addestrato (questa è la condizione sperimentale vera e propria);

2. con f = una rete della stessa architettura ma con pesi scelti a caso, mai addestrata: se l'effetto di additività sopravvive anche qui, vuol dire che l'origine è l'aggregazione e non l'addestramento;
3. nello spazio delle feature statistiche grezze (per esempio quelle calcolate dallo strumento CICFlowMeter) — attenzione, molte di queste feature sono conteggi e somme di eventi, quindi additive quasi per definizione: questo controllo da solo può far crollare l'intera ipotesi, ed è per questo importante includerlo;
4. nello spazio della distanza di compressione normalizzata (NCD) calcolata sui byte grezzi del traffico, un metodo che non richiede alcun addestramento né alcun modello.

Metriche. Non basarsi sulla sola cosine similarity, perché ignora la lunghezza (norma) dei vettori e quindi può nascondere componenti costanti condivise fra tutte le esecuzioni (nel nostro caso, per esempio, il traffico del canale di comando e controllo, che è identico in tutte le esecuzioni e può dominare il segnale mascherando l'effetto reale). Conviene invece usare il residuo relativo $\|\Delta(z_3) - \Delta(z_1) - \Delta(z_2)\| / \|\Delta(z_3)\|$, dove $\Delta(z) = f(g(z)) - f(g(\emptyset))$ è la rappresentazione a cui si è sottratta quella di un'operazione "vuota" (nessuna azione), presa come riferimento neutro. È utile riportare separatamente anche il rapporto fra le norme dei vettori coinvolti. La distribuzione nulla, cioè il termine di paragone contro cui giudicare se un risultato è sorprendente o no, si costruisce con un test di permutazione su coppie di tecniche scelte a caso (si veda il glossario).

Difficoltà. Media. La parte laboriosa è l'allestimento pratico del laboratorio Caldera e la randomizzazione accurata di tutti gli artefatti sperimentali (indirizzi, tempi, ecc.); la parte matematica è la costruzione dei test statistici corretti, ma è alla portata di una tesi triennale con un minimo di supporto.

4.2 Tesi B — Quali tecniche ATT&CK sono linearmente decodificabili dal traffico?

Domanda. Data una rappresentazione di traffico y , per quali tecniche ATT&CK esiste una direzione lineare w tale che il prodotto scalare $\langle w, y \rangle$ predica la presenza di quella tecnica meglio di quanto farebbe il caso puro?

Perché è interessante. È una mappa del territorio: dice, con dati alla mano, dove ha senso continuare a cercare. Risponde in modo empirico all'ostacolo del § 3.5: definisce l'insieme delle tecniche su cui tutto il resto del programma di ricerca ha effettivamente senso, e chiunque voglia lavorare in quest'area dopo di noi può ripartire da questi risultati. È anche il tipo di lavoro che si presenta bene in una tesi: produce naturalmente una tabella, una figura, un elenco ordinato di tecniche per grado di decodificabilità.

Esperimento. Si addestra un probe lineare per ciascuna tecnica (un piccolo classificatore lineare, si veda il glossario) su esecuzioni singole della tecnica, e lo si valuta su esecuzioni indipendenti mai viste durante l'addestramento del probe. Si confrontano rappresentazioni di diversa natura: modello fondazionale congelato (cioè non riaddestrato), IDS supervisionato, feature statistiche classiche, byte grezzi. Si analizza infine quali proprietà della tecnica (protocollo di rete usato, durata, volume di traffico, numero di host coinvolti) predicono meglio la decodificabilità.

Metriche. L'AUC (Area Under the Curve, si veda il glossario) del probe, confrontata con una soglia

nulla ottenuta permutando casualmente le etichette; la curva di apprendimento (quante esecuzioni servono prima che le prestazioni smettano di migliorare); la robustezza al cambiare delle condizioni di rete.

Difficoltà. Bassa-media. È l'opzione più sicura del gruppo: produce risultati presentabili anche nello scenario in cui l'ipotesi principale dell'intero filone di ricerca risultasse falsa.

4.3 Tesi C — Invarianza all'ordine e sue conseguenze di sicurezza

Domanda. La rappresentazione calcolata da un IDS distingue due campagne d'attacco che usano lo stesso insieme di tecniche ma in ordine diverso?

Perché è interessante. È una domanda che, per come è costruita, non può restituire un risultato "vuoto" o inconcludente — e vale la pena capire perché, perché progettare esperimenti così è una delle competenze più preziose che si possano portare via da una tesi. Ci sono due soli esiti possibili, ed entrambi sono risultati scientificamente validi e pubblicabili:

- Se la varianza fra ordini diversi è grande rispetto alla varianza fra ripetizioni dello stesso ordine, allora la rappresentazione conserva davvero l'informazione sull'ordine di esecuzione, l'additività fallisce, e si è riusciti a misurare quantitativamente il problema del binding (§ 3.3) in un dominio reale, non solo in teoria.
- Se invece la varianza fra ordini diversi è indistinguibile dal semplice rumore statistico, la rappresentazione è cieca all'ordine di esecuzione. Ma allora un attaccante potrebbe permutare liberamente le fasi del proprio attacco senza alcun effetto sul rilevatore: avremmo scoperto una vera e propria falla di evasione, che è a tutti gli effetti un risultato di sicurezza informatica.

Esperimento. Si fissano insiemi di k tecniche (con $k = 2$ oppure 3), si eseguono tutte le $k!$ (k fattoriale, cioè tutte le possibili sequenze di ordine: per $k=3$ sono 6) permutazioni possibili di quell'insieme, ciascuna ripetuta r volte per avere una stima affidabile. Si calcola poi la varianza fra ordini diversi e la varianza fra ripetizioni dello stesso ordine.

Metriche. Un rapporto in stile analisi della varianza (la quota di varianza spiegata dall'ordine di esecuzione, rispetto alla varianza residua non spiegata), corredato da intervalli di confidenza calcolati con la tecnica del bootstrap (si veda il glossario). Come conferma indipendente: si addestra un classificatore che, data solo la rappresentazione finale, provi a indovinare l'ordine di esecuzione; la sua accuratezza è una misura diretta di quanta informazione sull'ordine sopravvive davvero nella rappresentazione.

Difficoltà. Media. Il costo è soprattutto combinatorio nelle esecuzioni di laboratorio necessarie (il numero di permutazioni cresce molto in fretta con k), quindi conviene tenere k piccolo per restare nei tempi di una tesi.

4.4 Tesi D — Il fine-tuning supervisionato distrugge la geometria compositiva?

Domanda. Durante l'addestramento supervisionato di un rilevatore, la struttura compositiva (cioè la capacità di scomporre le rappresentazioni in "pezzi" corrispondenti a concetti distinti) presente nelle rappresentazioni pre-addestrate aumenta o diminuisce, e con quale andamento nel

tempo?

Perché è interessante. Perché mette in tensione due risultati già pubblicati in letteratura, che a prima vista sembrano in contraddizione. Da un lato, Vaze, Han, Vedaldi e Zisserman (2022, *"Open-Set Recognition: a Good Closed-Set Classifier is All You Need?"*, riferimento [4] in bibliografia, § 6) mostrano che migliorare l'accuratezza su un problema "chiuso" (con classi note in anticipo) migliora anche, come effetto collaterale, la capacità di riconoscere ciò che è sconosciuto. Dall'altro lato, il fenomeno del neural collapse (§ 3.4) dice che portare un classificatore a convergenza riduce la rappresentazione a un rango di sole $K - 1$ dimensioni utili, distruggendo ogni struttura più fine. I due risultati non possono essere entrambi veri senza ulteriori precisazioni, e capire quale dei due prevale nel dominio specifico del traffico di rete è una domanda ben posta a cui, per quanto ne sappiamo, nessuno ha ancora risposto in letteratura.

Esperimento. Si parte da un modello fondazionale già pre-addestrato e congelato; si esegue il fine-tuning (il riaddestramento mirato, si veda il glossario) su un compito di rilevamento specifico; si salvano checkpoint (istantanee del modello) a intervalli regolari durante questo processo. Su ciascun checkpoint si misura: (a) l'accuratezza sul compito di classificazione chiuso; (b) la dimensione effettiva della rappresentazione (tramite il rango numerico o l'entropia degli autovalori della matrice di covarianza, si veda il glossario); (c) la qualità dei probe lineari per tecnica descritti al § 4.2; (d) il residuo di additività calcolato come al § 4.1.

Metriche. Le quattro curve elencate sopra, tracciate in funzione del passo di addestramento (cioè di quanto lontano si è arrivati nel fine-tuning). La domanda pratica diventa: esiste un punto nel training in cui (a) è già soddisfacente, ma (b), (c) e (d) non si sono ancora degradate troppo? Se sì, si è identificato un regime operativo utile — un punto d'arresto consigliato dell'addestramento — che è un risultato di valore pratico immediato.

Difficoltà. Medio-alta sul piano computazionale (serve addestrare e salvare molti checkpoint), ma è la proposta concettualmente più elegante del gruppo, e si può realizzare anche con modelli di dimensioni contenute, alla portata di risorse da tesi triennale.

4.5 Tesi E — Un banco di prova per la novità compositiva

Domanda. Un rilevatore addestrato solo su tecniche singole (mai viste in combinazione) riconosce combinazioni mai viste di quelle stesse tecniche? E soprattutto: come si separa questa capacità dalla semplice sensibilità a un cambiamento delle condizioni generali di rete, che non ha nulla a che fare con la novità dell'attacco in sé?

Perché è interessante. Perché il campo, a oggi, non dispone di un banco di prova (benchmark) che separi chiaramente la novità semantica (un attacco davvero nuovo nella sua logica) dalla novità di contesto (la stessa identica cosa osservata su una rete diversa, con condizioni diverse) — e questa confusione è probabilmente alla radice del fatto che molti risultati promettenti in laboratorio non sopravvivono quando applicati al mondo reale. Il modello da imitare esiste già nel campo della visione artificiale: il Semantic Shift Benchmark introdotto nello stesso lavoro di Vaze e colleghi [4], che stratifica (cioè suddivide in fasce) le classi non viste in "facili" e "difficili" in base alla loro somiglianza semantica con le classi già viste in addestramento.

Esperimento. Si costruisce una suddivisione stratificata dei dati in cinque gruppi: viste = tecniche singole già note; non viste facili = combinazioni di tecniche appartenenti a tattiche diverse fra

loro; non viste difficili = combinazioni interne alla stessa tattica, quindi massimamente simili e confondibili con le tecniche singole già note; non viste nuove = tecniche tenute completamente fuori dall'addestramento; controllo di disturbo = tecniche già viste, ma osservate sotto condizioni di rete alterate (più traffico di sottofondo, latenza diversa, dimensione massima dei pacchetti diversa). Quest'ultimo gruppo di controllo è quello che di solito manca negli studi pubblicati, e che serve invece a tutti per separare i due tipi di novità.

Metriche. L'AUROC (si veda il glossario) calcolata separatamente per ciascuno strato descritto sopra. Una baseline (un metodo di riferimento semplice contro cui confrontarsi) è obbligatoria: lo stesso lavoro di Vaze e colleghi mostra che un punteggio semplicissimo, ricavato dal logit massimo di un buon classificatore ordinario, compete alla pari con metodi molto più elaborati nel riconoscimento dell'ignoto. Se il metodo compositivo proposto non riesce a battere questa baseline così semplice, il risultato onesto da riportare è che, in questo caso, il metodo più complesso non serve.

Difficoltà. Alta come mole di lavoro sperimentale da svolgere, ma bassa come rischio di non ottenere alcun risultato pubblicabile. Adatta a chi preferisce la costruzione accurata e paziente di un manufatto sperimentale solido, piuttosto che la scoperta di un fenomeno nuovo e incerto.

4.6 Come orientarsi fra le cinque

Nessuna delle cinque proposte è "la migliore" in assoluto: dipende da cosa si desidera portarsi via dall'esperienza della tesi.

- Se il criterio è la massima probabilità di arrivare in fondo con un risultato pulito e presentabile: Tesi B, poi Tesi C.
- Se il criterio è il massimo contenuto matematico: Tesi D (algebra lineare e analisi spettrale) e Tesi A (statistica inferenziale, costruzione di distribuzioni nulle, test di permutazione).
- Se il criterio è la rilevanza diretta per la sicurezza informatica pratica: Tesi C, che qualunque sia l'esito produce comunque un'affermazione concreta su cosa un attaccante può o non può fare per eludere il rilevatore.
- Se si preferisce costruire con cura un manufatto sperimentale solido piuttosto che rincorrere una scoperta incerta: Tesi E.

Un avvertimento pratico che vale per le tesi A, B, C ed E: hanno tutte un costo fisso di allestimento del laboratorio Caldera, che è la voce di lavoro che più spesso fa slittare le tempistiche di una tesi triennale. Vale la pena stimarla con calma insieme al relatore prima di scegliere, e considerare eventualmente di ridurre l'ambizione dell'esperimento in cambio della certezza di riuscirci nei tempi previsti. La Tesi D è l'unica che si può realizzare quasi interamente a partire da dati già disponibili, senza dover allestire un laboratorio da zero.

5 Glossario

In ordine di apparizione approssimativo. Le voci sono volutamente brevi e pensate per chi non ha alcun background di machine learning: servono a poter proseguire la lettura senza doversi fermare a ogni termine tecnico, non a sostituire uno studio più approfondito della materia.

Machine learning (apprendimento automatico)

L'insieme di tecniche con cui un programma impara a svolgere un compito a partire da esempi, invece di essere programmato passo per passo da un umano.

Rete neurale

Un tipo di programma ispirato (molto alla lontana) al funzionamento del cervello, composto da molte unità di calcolo semplici (i neuroni) collegate fra loro, i cui parametri vengono aggiustati durante l'addestramento in base agli esempi mostrati.

Addestramento (training)

Il processo con cui una rete neurale aggiusta progressivamente i propri parametri interni guardando molti esempi, in modo da ridurre via via l'errore che commette.

Gradient descent (discesa del gradiente)

La tecnica matematica più comune con cui, durante l'addestramento, i parametri della rete vengono corretti un po' alla volta, sempre nella direzione che riduce l'errore commesso sugli esempi visti finora.

Rappresentazione (embedding, stato nascosto)

Il vettore di numeri reali (cioè la lista ordinata di numeri) in cui una rete neurale trasforma un input a un certo punto della sua elaborazione. È tutto ciò che la rete "sa" dell'input in quel punto, e non conserva l'input originale.

Vettore

In questo documento, semplicemente una lista ordinata di numeri. Un vettore con 500 numeri si dice "a 500 dimensioni".

Neurone

La più piccola unità di calcolo di una rete neurale; molti neuroni collegati insieme formano la rete.

Rappresentazione distribuita

Codifica in cui un concetto non corrisponde a un singolo neurone ma a un pattern di attivazione su molti neuroni contemporaneamente, ciascuno dei quali partecipa a più concetti diversi.

Interpretabilità

Il campo di ricerca che studia come "leggere" e capire cosa rappresenta davvero lo stato interno di una rete neurale, invece di trattarla come una scatola nera che produce solo un output finale.

Superposizione

Il fatto che una rete rappresenti più concetti di quante siano le sue dimensioni disponibili, sovrappo-
nendoli linearmente lungo direzioni che non sono perpendicolari (ortogonali) fra loro.

Funzione lineare

Una funzione che si comporta "in modo semplice" rispetto a somme e moltiplicazioni: sommare gli input equivale a sommare gli output, e moltiplicare l'input per un numero moltiplica l'output per lo stesso numero. È l'opposto di una funzione non lineare, il cui comportamento può essere arbitrariamente complicato e imprevedibile.

Ipotesi di rappresentazione lineare

L'ipotesi secondo cui i concetti corrispondono a direzioni nello spazio delle rappresentazioni di una rete, e si combinano fra loro per semplice somma.

Additività

La proprietà per cui la rappresentazione della somma di due concetti è (circa) uguale alla somma delle rappresentazioni dei due concetti singoli.

Variabile latente

Una quantità non osservata direttamente, che si suppone abbia contribuito a generare il dato osservato. Nel nostro caso: la presenza (o assenza) di una particolare tecnica d'attacco.

Vettore sparso, sparsità

Un vettore sparso è fatto quasi tutto di zeri, con solo poche componenti diverse da zero. La sparsità è, appunto, questa proprietà.

Identificabilità

La proprietà per cui le variabili latenti vere si possono recuperare a partire dai soli dati osservati, eventualmente a meno di ambiguità note e accettate (per esempio una permutazione dell'ordine, o un fattore di scala).

ICA (Independent Component Analysis, analisi delle componenti indipendenti)

Famiglia di metodi statistici che separano un segnale osservato composito nelle sue componenti originarie, assumendo che siano statisticamente indipendenti fra loro. È l'antenato diretto di gran parte della teoria descritta in questo documento.

Compressed sensing (campionamento compresso)

Teoria matematica che stabilisce quando e come un segnale sparso possa essere ricostruito a partire da un numero di misure molto inferiore alla sua dimensione nominale.

Sparse coding (codifica sparsa)

Procedura che rappresenta un dato come combinazione di poche colonne di un "dizionario" appreso, minimizzando insieme l'errore di ricostruzione e il numero di componenti diverse da zero usate (la norma ℓ_1 dei coefficienti).

Sparse autoencoder (SAE)

Attenzione, il termine ha due significati opposti a seconda del contesto. (1) Nel campo dell'interpretabilità: una rete che mappa una rappresentazione in un vettore di codici sparso e sovra-completo (cioè più grande dell'input), usata per estrarre concetti leggibili. (2) Nella letteratura classica sugli IDS, da vent'anni: un autoencoder sotto-completo (con uno strato interno più piccolo dell'input) con una penalità di sparsità, usato per ridurre la dimensionalità dei dati. Sono, di fatto, cose opposte: prestare sempre attenzione a quale dei due significati è in gioco.

Autoencoder

Una rete neurale addestrata a ricostruire il proprio input in uscita, passando per uno strato interno più piccolo (sotto-completo) o più grande (sovra-completo) dell'input stesso; è un modo comune per imparare rappresentazioni utili senza bisogno di etichette.

Monosemanticità / polisemanticità

Un neurone è monosemantico se risponde in modo affidabile a un solo concetto interpretabile; è polisemantico se invece mescola la risposta a diversi concetti scorrelati fra loro.

Problema del binding

Il fatto che una rappresentazione puramente additiva non possa codificare quale attributo appartenga a quale oggetto, né in quale ordine gli eventi siano avvenuti: la somma è per sua natura invariante a permutazioni e associazioni.

Pooling

Operazione che aggrega molte rappresentazioni distinte in una sola, tipicamente calcolandone la media. È l'operazione che, al § 3.1, rischia di rendere il risultato di addittività vero "per costruzione" anziché per merito della rete.

Cosine similarity (similarità del coseno)

Il coseno dell'angolo formato da due vettori. Misura quanto due vettori "puntano nella stessa direzione", ignorando completamente la loro lunghezza (norma). Vale 1 se sono paralleli e concordi, 0 se sono perpendicolari, -1 se sono opposti.

Norma di un vettore

Una misura della "lunghezza" o "grandezza" di un vettore, distinta dalla sua direzione.

Anisotropia

Proprietà di uno spazio di rappresentazione in cui i vettori si concentrano in una zona ristretta

(un "cono stretto") invece di distribuirsi uniformemente in tutte le direzioni, cosicché due elementi qualsiasi appaiono già molto simili fra loro anche senza alcuna relazione reale. Rende indispensabile l'uso di baseline (termini di paragone) calibrate con cura.

Neural collapse (collasso neurale)

Fenomeno per cui, nella fase finale dell'addestramento di un classificatore, le rappresentazioni collassano verso una configurazione geometrica molto semplice (un simpleso) con tante direzioni quante sono le classi da distinguere, azzerando quasi del tutto la variabilità interna a ciascuna classe.

Rango (di una matrice o di un insieme di vettori)

Il numero di dimensioni "davvero indipendenti" occupate da un insieme di vettori, che può essere molto minore del numero grezzo di componenti che li descrivono.

Probe lineare

Un piccolo classificatore lineare, cioè estremamente semplice, addestrato sopra una rappresentazione già calcolata e "congelata" (non più modificata), usato per misurare quanta informazione su una certa proprietà quella rappresentazione contenga in una forma facilmente accessibile (linearmente).

Fine-tuning

Il riaddestramento mirato di un modello già pre-addestrato, applicato a un compito specifico più ristretto, invece di addestrare un modello completamente da zero.

Modello fondazionale (foundation model)

Un modello di grandi dimensioni, pre-addestrato in modo auto-supervisionato su un'enorme quantità di dati non etichettati, che viene poi adattato (con il fine-tuning) a compiti diversi e specifici. Ne esistono ormai versioni pensate apposta per il traffico di rete (per esempio ET-BERT, netFound, TrafficFormer).

Auto-supervisionato

Un modo di addestrare un modello senza bisogno di etichette scritte a mano da un umano: il compito di addestramento viene ricavato automaticamente dai dati stessi (per esempio, "indovina la parola mancante in questa frase").

Zero-shot

La valutazione di un modello su casi di un tipo che non ha mai visto durante l'addestramento.

Riconoscimento a insieme aperto (open-set recognition)

Il problema di far riconoscere a un modello che un campione osservato non appartiene a nessuna delle classi note ("nessuna delle precedenti"), invece di forzarlo a scegliere comunque una delle categorie già viste.

Logit

Il punteggio numerico grezzo prodotto dall'ultimo strato di un classificatore, prima che venga trasformato in una probabilità tramite la funzione softmax. La sua grandezza (magnitudine) porta informazione utile che la softmax, normalizzando tutto fra 0 e 1, in parte cancella.

Softmax

Una funzione che trasforma un insieme di punteggi grezzi (i logit) in un insieme di probabilità che sommano a 1, tipicamente usata come ultimo passo di un classificatore.

AUC / AUROC (Area Under the [Receiver Operating Characteristic] Curve)

L'area sotto la curva ROC: rappresenta la probabilità che il modello assegni un punteggio più alto a un esempio positivo scelto a caso rispetto a un esempio negativo scelto a caso. Vale 0,5 per un classificatore che decide a caso, 1 per un classificatore perfetto.

Baseline

Un metodo semplice, spesso banale, contro cui si confrontano i risultati del proprio metodo più sofisticato. Un esperimento senza baseline appropriate non dimostra, di fatto, nulla: senza un termine di paragone non si può dire se il metodo proposto sia davvero utile.

Test di permutazione

Un modo non parametrico (cioè che non assume nulla sulla forma dei dati) di costruire un termine di paragone statistico: si rimescolano ripetutamente e a caso le etichette dei dati, e si osserva quanto spesso il puro caso produce, da solo, un effetto grande quanto quello effettivamente misurato.

Bootstrap

Una tecnica statistica per stimare quanto un risultato potrebbe variare per puro effetto del campionamento, ricampionando ripetutamente e con reinserimento i dati a disposizione.

Distribuzione nulla

Il termine di paragone statistico rispetto al quale si giudica se un risultato osservato è sorprendente oppure spiegabile dal solo caso.

NCD (Normalized Compression Distance, distanza di compressione normalizzata)

Una misura di somiglianza fra due oggetti basata su quanto meglio si riescono a comprimere insieme rispetto a comprimerli separatamente. Non richiede alcun addestramento né parametri, ed è quindi un ottimo termine di paragone "neutro".

IDS (Intrusion Detection System, sistema di rilevamento delle intrusioni)

Un sistema, spesso basato su machine learning, che analizza il traffico di rete o gli eventi di un sistema informatico per segnalare attività potenzialmente ostili.

pcap / flusso

Un pcap è una cattura grezza e completa dei singoli pacchetti di rete. Un flusso è invece l'aggregazione di tutti i pacchetti scambiati fra due estremi nel corso di una connessione, riassunta tramite statistiche (durata, numero di byte, ecc.) invece di conservare ogni singolo pacchetto.

MITRE ATT&CK

Una tassonomia pubblica dei comportamenti degli attaccanti informatici, organizzata in tattiche (l'obiettivo perseguito) e tecniche (il modo concreto per raggiungerlo).

Caldera

Una piattaforma open source di emulazione dell'avversario, che esegue realmente tecniche ATT&CK all'interno di un laboratorio controllato, generando traffico di rete etichettato con la tecnica corrispondente. Dal 2026 è un progetto Apache in fase di incubazione.

Semantic gap (divario semantico)

Il divario fra ciò che un rilevatore automatico segnala in modo tecnico e sintetico, e ciò che una persona ha davvero bisogno di sapere per decidere come agire. È uno dei motivi storici per cui i rilevatori basati su machine learning faticano a essere adottati concretamente sul campo.

6 Riferimenti bibliografici

Elenchiamo qui i quattro lavori citati nel testo, con titolo completo, in modo che sia possibile trovarli facilmente cercando il titolo per esteso invece che i soli autori e l'anno.

- [1] Klindt, D., O'Neill, C., Reizinger, P., Maurer, H., Miolane, N. (2025). *"From Superposition to Sparse Codes: Interpretable Representations in Neural Networks"*. arXiv:2503.01824. È il riferimento principale per tutto il quadro concettuale delle sezioni 1.2 e 1.3 (ipotesi di rappresentazione lineare, superposizione, identificabilità, sparse coding, interpretabilità quantitativa): la lettura più utile per chi voglia approfondire prima di scegliere una tesi.
- [1] Papayan, V., Han, X. Y., Donoho, D. L. (2020). *"Prevalence of Neural Collapse during the Terminal Phase of Deep Learning Training"*. Proceedings of the National Academy of Sciences (PNAS), 117(40), 24652–24663. Anche su arXiv:2008.08186. Il riferimento per il fenomeno del neural collapse citato al § 3.4.
- [1] Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K. (2022). *"Dos and Don'ts of Machine Learning in Computer Security"*. 31st USENIX Security Symposium (USENIX Security '22), pp. 3971–3988. Premio Distinguished Paper Award al medesimo convegno. Il riferimento per l'igiene sperimentale discussa al § 3.6.
- [1] Vaze, S., Han, K., Vedaldi, A., Zisserman, A. (2022). *"Open-Set Recognition: a Good Closed-Set Classifier is All You Need?"*. International Conference on Learning Representations (ICLR 2022, presentazione orale). Anche su arXiv:2110.06207. Il riferimento per il Semantic Shift

Benchmark e per la baseline del logit massimo discussi alle Tesi D ed E (§ 4.4, § 4.5).